

Sudoku Genérico como CSP

Este notebook gera e resolve um Sudoku $n^2 \times n^2$ (com n parametrizável) modelado como um **problema de satisfação de restrições (CSP)**.

A ideia central

Todas as regras do Sudoku são, no fundo, **a mesma regra**:

Um conjunto de células tem de ter valores todos diferentes.

O que muda entre uma linha, uma coluna e um bloco é apenas **quais células** pertencem ao conjunto. Por isso, em vez de escrever três tipos de restrição, defino **uma só abstração** (`Box`) e construo tudo a partir dela.

Elemento	Papel	Como é construído
<code>Box</code>	Grupo genérico de células ("todos diferentes")	classe base
<code>Cube</code>	Bloco $n \times n$	especialização de <code>Box</code>
<code>Path</code>	Linha ou coluna (troço reto)	especialização de <code>Box</code>
Pistas	Células pré-preenchidas aleatoriamente	simples dicionário <code>{(i, j): valor}</code>

Estrutura do notebook

1. **Classes** — `Box`, `Cube`, `Path`
2. **Modelo e resolução** — `CriaPistas`, `ResolveSudoku`, `MostraSudoku`
3. **Validação** — `sudokuIsValido`
4. **Teste** — gerar, resolver e validar 10 puzzles

/// attention | Antes de correr O parâmetro `n` está definido na primeira célula de código.
Para o Sudoku clássico 9×9 usa `n = 3`; para uma grelha 4×4 usa `n = 2`. ///

```
In [ ]: import marimo as mo
import random
from ortools.sat.python import cp_model

n = 10

class Box:
    def __init__(self, celulas):
        if isinstance(celulas, dict): #Verifica se celulas é um dicionario
```

```

        self.celulas = dict(celulas) #Se for um dicionario copia-o para o para si mesmo
    else:
        self.celulas = {c: None for c in celulas} #Caso nao seja um dicionario cria um dicionario que a cada celula c associa um None

    def add(self, i, j, val=None):
        if not (0 <= i < n**2 and 0 <= j < n**2):
            raise ValueError(f"Célula ({i}, {j}) fora da grelha")
        if val is not None and not (1 <= val <= n**2):
            raise ValueError(f"Valor {val} fora de 1..{n**2}")
        self.celulas[(i, j)] = val

    def matriz(self):
        N = n**2
        m = [[0] * N for _ in range(N)]
        for (i, j), val in self.celulas.items(): # o celulas.items() retorna os pares chave,valor
            if val is not None:
                m[i][j] = val
        return m

class Cube(Box):
    def __init__(self, i, j):
        if not (0 <= i < n and 0 <= j < n):
            raise ValueError(f"Bloco ({i}, {j}) fora da grelha de blocos")
        celulas = {
            (i * n + l, j * n + c)
            for l in range(n)
            for c in range(n)
        }
        super().__init__(celulas)

class Path(Box):
    def __init__(self, inicio, fim):
        i1, i2 = inicio
        f1, f2 = fim
        if not all(0 <= x < n**2 for x in (i1, i2, f1, f2)):
            raise ValueError("Início ou fim fora da grelha")
        if i1 != f1 and i2 != f2:
            raise ValueError("Não existe caminho reto entre as posições")
        if i1 > f1: # coluna constante, a subir
            celulas = {(f1 + l, f2) for l in range(i1 - f1 + 1)}
        elif i1 < f1: # coluna constante, a descer
            celulas = {(i1 + l, i2) for l in range(f1 - i1 + 1)}
        elif i2 > f2: # linha constante, para a esquerda
            celulas = {(f1, f2 + c) for c in range(i2 - f2 + 1)}
        elif i2 < f2: # linha constante, para a direita
            celulas = {(i1, i2 + c) for c in range(f2 - i2 + 1)}
        else: # inicio == fim: uma só célula
            celulas = {(i1, i2)}
        super().__init__(celulas)

```

1. As classes: Box, Cube e Path

Box — o grupo genérico

Um `Box` guarda um **dicionário** `{(linha, coluna): valor}`, onde:

- `valor = None` → a célula pertence ao grupo mas está **livre**;
- `valor = inteiro` → a célula está **fixa** (pista) a esse valor.

Escolhi um dicionário porque só ocupa memória para as células que realmente pertencem ao grupo (uma linha tem n^2 células, não n^4) e permite aceder a qualquer célula em tempo constante.

Métodos:

- `add(i, j, val=None)` — acrescenta a célula (i, j) ao grupo. Levanta `ValueError` se as coordenadas estiverem fora de $[0, n^2)$ ou se o valor estiver fora de $[1, n^2]$.
- `matriz()` — devolve o grupo como matriz $n^2 \times n^2$, com `0` nas células livres ou que não pertencem ao grupo, e o valor fixo nas restantes.

O `Box` **não sabe nada** sobre linhas, colunas ou blocos. É esta ignorância que o torna reutilizável.

Cube — bloco $n \times n$

`Cube(i, j)` representa o bloco de índices (i, j) , com $0 \leq i, j < n$. O seu canto superior esquerdo é a célula $(i \cdot n, j \cdot n)$ e contém as n^2 células

$$\{(i \cdot n + l, j \cdot n + c) \mid 0 \leq l, c < n\}$$

Path — troço reto

`Path(inicio, fim)` devolve todas as células entre duas coordenadas **inclusive**, desde que estejam na mesma linha ou na mesma coluna. Funciona nos dois sentidos (`fim` depois ou antes de `inicio`), e se `inicio == fim` devolve uma única célula. É assim que se constroem as linhas e as colunas:

```
Path((i, 0), (i, N - 1)) # linha i
Path((0, j), (N - 1, j)) # coluna j
```

/// note | Herança `Cube` e `Path` limitam-se a **calcular o conjunto de células** e a passá-lo ao construtor de `Box` através de `super().__init__`. Não repetem nenhuma lógica de restrição. ///

```
In [ ]: def CriaPistas():
        clues = n
```

```

N = n**2
todasCords = [(i,j) for i in range(N) for j in range(N)]
celulas = random.sample(todasCords , clues) # cria celulas em sitios
aleatorios
    return {c: random.randint(1, N) for c in celulas} # associa um val aleatorio
ao dicionario com feito na linha anterior

def ResolveSudoku(pistas):
    N = n**2
    model = cp_model.CpModel() # cria o model

    x = {(i, j): model.NewIntVar(1, N, f"x_{i}_{j}") # associa um valor int a cada
    cordenada i , j
        for i in range(N) for j in range(N)}

    # linhas
    for i in range(N):
        g = Path((i, 0), (i, N - 1)) # cria um dicionario de coodenadas que
        representa cada linha do tabuleiro
        model.AddAllDifferent([x[pos] for pos in g.celulas]) # restringe o valor
        das linhas para serem diferentes por linha

    # colunas
    for j in range(N):
        g = Path((0, j), (N - 1, j))
        model.AddAllDifferent([x[pos] for pos in g.celulas])

    # cubos
    for i in range(n):
        for j in range(n):
            g = Cube(i, j)
            model.AddAllDifferent([x[pos] for pos in g.celulas])

    # pistas: dicionário {(i, j): valor}
    for pos, val in pistas.items(): # o .items percorre o dicionario retornando um
    par (chave, valor)
        model.Add(x[pos] == val)

    solver = cp_model.CpSolver()
    estado = solver.Solve(model)

    if estado in (cp_model.OPTIMAL, cp_model.FEASIBLE):
        return {pos: solver.Value(var) for pos, var in x.items()}
    return None

def MostraSudoku(solucao, pistas=None): # função feita com Llm
    """Imprime a grelha resolvida.
    solucao: dict {(i, j): valor} devolvido por ResolveSudoku (ou None).
    pistas: dict opcional {(i, j): valor}; as células de pista aparecem entre [
    ].
    """
    if solucao is None:
        print("Sem solução.")
        return

    N = n**2

```

```

pistas = pistas or {}
larg = len(str(N)) # largura de cada célula (1 p/ 9x9, 2 p/ 16x16...)
bloco = n * (larg + 2) + 1 # largura de um bloco em caracteres
sep = "+" + "+" * bloco * n + "+"

for i in range(N):
    if i % n == 0:
        print(sep)
    partes = []
    for b in range(n):
        celulas = []
        for j in range(b * n, (b + 1) * n):
            v = str(solucao[(i, j)]).rjust(larg)
            celulas.append(f"[{v}]" if (i, j) in pistas else f"{v} ")
        partes.append("".join(celulas))
    print("|" + "|".join(partes) + "|")
print(sep)

```

2. Pistas, modelo CSP e apresentação

CriaPistas() — pistas aleatórias

Escolhe $k = n$ células distintas da grelha (com `random.sample`, que não repete) e atribui a cada uma um valor aleatório em $[1, n^2]$. O resultado é um simples dicionário `{(i, j): valor}`, no mesmo formato interno de um `Box`.

ResolveSudoku(pistas) — o modelo CSP

Resolvo o problema com o **CP-SAT do OR-Tools**, um solver de programação por restrições que combina propagação de restrições com pesquisa e aprendizagem de cláusulas (SAT). Escolhi-o por ser a sugestão da disciplina e por tratar muito bem restrições `AllDifferent`.

O modelo tem três ingredientes:

Componente	Definição
Variáveis	$x_{i,j} \in [1, n^2]$, uma por célula
Restrições	<code>AllDifferent</code> sobre cada linha, coluna e bloco
Pistas	$x_{i,j} = v$ para cada $(i, j) \mapsto v$ das pistas

As linhas e colunas são construídas com `Path` e os blocos com `Cube`; o modelo só olha para o conjunto de células de cada grupo (`g.celulas`), nunca para a sua origem. Isto significa que acrescentar uma diagonal ou uma região irregular exigiria apenas mais um grupo, sem tocar no resto.

Se o solver encontra solução (`OPTIMAL` ou `FEASIBLE`), a função devolve um dicionário `{(i, j): valor}` ; caso contrário devolve `None` , o que permite distinguir claramente "sem solução" de uma grelha válida.

```
/// warning | Pistas aleatórias podem ser contraditórias Como as pistas são sorteadas sem
verificar compatibilidade, duas delas podem violar as regras (por exemplo, o mesmo valor
duas vezes na mesma linha). Nesse caso o puzzle não tem solução, e ResolveSudoku
devolve None . Optei por reportar o insucesso em vez de gerar novas pistas até haver
solução, para manter a função simples e deixar visível a frequência com que isto acontece.
///
```

MostraSudoku(solucao, pistas) — apresentação

Imprime a grelha em texto, separando os blocos com `+---+` e `|` . A largura de cada célula adapta-se a n (1 dígito em 9×9 , 2 dígitos em 16×16), e as **células de pista aparecem entre parênteses retos**, por exemplo `[5]` , para se distinguirem das preenchidas pelo solver. (Função escrita com apoio de um LLM.)

```
In [ ]: def sudokuIsValido(sudoku):
        if sudoku is None:
            return False
        if not (checkCubes(sudoku) and checkPath(sudoku)):
            print("O sudoku não é valido")
            return False
        print ('O sudoku é valido')
        return True

def valoresOk(sudoku, grupo):
    N = n**2
    valores = []
    for pos in grupo.celulas:
        val = sudoku.get(pos) # o .get retorna o valor associado a uma chave sem
        dar erro caso a chave nao exista no dicionario
        if val is None or not (1 <= val <= N):
            return False
        valores.append(val) # adiciona val á lista
    return len(valores) == len(set(valores))

def checkCubes(sudoku):
    return all(valoresOk(sudoku, Cube(i, j)) # o all retorna True caso todos as
    condicoes forem verdadeiras , neste caso verifica os valores do sudoku e os cubes
    todos

        for i in range(n) for j in range(n))

def checkPath(sudoku):
    N = n**2
    linhas = [Path((i, 0), (i, N - 1)) for i in range(N)]
    colunas = [Path((0, j), (N - 1, j)) for j in range(N)]
    return all(valoresOk(sudoku, g) for g in linhas + colunas)
```

3. Validação independente

Para não confiar cegamente no solver, a solução é verificada por código que **não usa o modelo CSP**. As funções reutilizam `Cube` e `Path` para enumerar os grupos:

Função	O que verifica
<code>valoresOk(sudoku, grupo)</code>	Todas as células do grupo têm valor em $[1, n^2]$ e sem repetições (compara o número de valores com o número de valores distintos, via <code>set</code>)
<code>checkCubes(sudoku)</code>	<code>valoresOk</code> em todos os n^2 blocos
<code>checkPath(sudoku)</code>	<code>valoresOk</code> em todas as n^2 linhas e n^2 colunas
<code>sudokuIsValido(sudoku)</code>	Combina as anteriores; devolve <code>False</code> se a solução for <code>None</code>

Como cada grupo tem exatamente n^2 células e todos os valores estão em $[1, n^2]$, "sem repetições" equivale a "contém exatamente os valores $1 \dots n^2$ ".

/// tip | Porquê validar? Se o modelo tivesse um erro (por exemplo, esquecer as colunas), o solver devolveria uma grelha que *parece* certa. Uma verificação separada apanha esse tipo de bug. ///

```
In [ ]: for i in range(10):
        pistas = CriaPistas()
        sudoku = ResolveSudoku(pistas)
        MostraSudoku(sudoku , pistas)
        sudokuIsValido(sudoku)
```

4. Teste do fluxo completo

A célula acima corre **10 vezes** o fluxo completo:

```
CriaPistas → ResolveSudoku → MostraSudoku → sudokuIsValido
```

Em cada iteração gera-se um novo conjunto de pistas aleatórias, resolve-se, mostra-se a grelha (pistas entre `[ ]`) e valida-se o resultado. Quando as pistas são contraditórias aparece "Sem solução." e a validação devolve `False` .

Como nada no código está fixo a 9×9 (tudo depende de `n`), basta alterar `n` para testar outras dimensões, como `n = 2` (4×4).

<https://claude.ai/share/124761c8-f63b-4ae7-8aad-556c7ac01c87> conversa que fez os markdown

<https://claude.ai/share/6bad0821-72a5-4a24-b8fc-b30065e950ff> conversa que fez a função mostra matriz

<https://claude.ai/share/48f541b3-764a-4c94-bcd3-901bdea62779> conversa que deu a ideia das pistas aleatorias